

Лабораторная работа №5

Вероятностные алгоритмы проверки чисел на простоту

Кюнкриков Даниил Саналович

Содержание

1	Цель работы	5
2	Задание	6
3	Теоретическое введение	7
3.1	Тест Ферма	7
3.2	Тест Соловья—Штрассена	7
3.3	Тест Миллера—Рабина	8
4	Выполнение лабораторной работы	9
4.1	Результаты	13
5	Выводы	14
	Список литературы	15

Список иллюстраций

4.1	Результат работы программы	13
-----	--------------------------------------	----

Список таблиц

1 Цель работы

Цель работы - Реализовать на языке Python вероятностные тесты простоты и продемонстрировать их работу.

2 Задание

Поставленные задачи:

1. Реализовать тест Ферма.
2. Реализовать алгоритм вычисления символа Якоби.
3. Реализовать тест Соловья–Штрассена.
4. Реализовать тест Миллера–Рабина.
5. Провести тестирование работы программы на нескольких входных данных.

3 Теоретическое введение

Проверка чисел на простоту — ключевая задача теории чисел и криптографии. Для больших чисел на практике применяются вероятностные тесты, которые быстро находят «свидетеля составности», а при успехе дают вывод «вероятно простое» с управляемой вероятностью ошибки [1; 2].

3.1 Тест Ферма

Если p — простое и $\gcd(a, p) = 1$, то по малой теореме Ферма:

$$a^{p-1} \equiv 1 \pmod{p}.$$

Тест Ферма проверяет равенство для случайных оснований a . Однако существуют числа Кармайкла, которые проходят тест Ферма для многих оснований, поэтому на практике используют более сильные тесты [1; 2].

3.2 Тест Соловья—Штрассена

Тест использует критерий Эйлера и символ Якоби (a/n) :

$$a^{(n-1)/2} \equiv (a/n) \pmod{n}$$

для простого n . Несовпадение даёт свидетель составности. Для нечётного составного n доля «ложных» оснований не превышает $1/2$, поэтому вероятность ошибки не более 2^{-t} при t независимых испытаниях [2; 5].

3.3 Тест Миллера—Рабина

Пусть $n - 1 = 2^s d$, где d нечётно. Для случайного основания a вычисляется $a^d \bmod n$ и далее последовательность квадратов. Если ни разу не получено $-1 \bmod n$, то найден свидетель составности. Для нечётного составного n доля «ложных» оснований не более $1/4$, поэтому вероятность ошибки оценивается как 4^{-t} [1; 3; 4].

4 Выполнение лабораторной работы

Программная реализация Теста Ферма Листинге №1.

4.0.1 Листинг 1 — Тест Ферма

```
def fermat_test_single(n: int) -> str:
    a = random.randint(2, n - 2)
    r = pow(a, n - 1, n)
    return "possible prime" if r == 1 else "composite"

def run_feramn_test(n: int, t: int) -> str:
    if n < 5:
        return "input должен быть >= 5"
    if n % 2 == 0:
        return "composite even"
    for _ in range(t):
        if fermat_test_single(n) == "composite":
            return "composite"
    return "possible prime"
```

Программная реализация алгоритма высчитывания символа Якоби Листинге №2.

4.0.2 Листинг 2 — Символ Якоби

```
def jacobi(a: int, n: int) -> int:
    """Вычисление символа Якоби (a/n) для нечётного n > 0."""
    if n <= 0 or n % 2 == 0:
        raise ValueError("n должно быть положительным нечётным")
    a %= n
    result = 1

    while a != 0:
        # выносим степени двойки из a
        while a % 2 == 0:
            a //= 2
            r = n % 8
            if r in (3, 5):
                result = -result

        # квадратичная взаимность
        a, n = n, a
        if a % 4 == 3 and n % 4 == 3:
            result = -result

        a %= n

    return result if n == 1 else 0
```

Программная реализация Теста Соловья-Штрассена показана на Листинге №3(#solovay).

4.0.3 Листинг 3 — Тест Соловья–Штрассена

```
def solovay_strassen_single(n: int) -> str:
    a = random.randint(2, n - 2)
    r = pow(a, (n - 1) // 2, n)

    # быстрый отсев
    if r not in (1, n - 1):
        return "composite"

    s = jacobi(a, n)
    return "possible prime" if r == (s + n) % n else "composite"

def run_solovay_strassen(n: int, t: int) -> str:
    if n < 2:
        return "composite"
    if n in (2, 3):
        return "possible prime"
    if n % 2 == 0:
        return "composite"

    for _ in range(t):
        if solovay_strassen_single(n) == "composite":
            return "composite"
    return "possible prime"
```

Программная реализация теста Миллера-Рабина показана на Листинге №4(#mr).

4.0.4 Листинг 4 — Тест Миллера–Рабина

```
def miller_rabin_single(n: int) -> str:
    # разложение  $n-1 = 2^s \cdot d$ 
    s = 0
    d = n - 1
    while d % 2 == 0:
        s += 1
        d //= 2

    a = random.randint(2, n - 2)
    y = pow(a, d, n)

    if y in (1, n - 1):
        return "possible prime"

    for _ in range(s - 1):
        y = pow(y, 2, n)
        if y == n - 1:
            return "possible prime"
        if y == 1:
            return "composite"
    return "composite"

def run_miller_rabin(n: int, t: int) -> str:
    if n < 2:
        return "composite"
```

```
if n in (2, 3):  
    return "possible prime"  
if n % 2 == 0:  
    return "composite"  
  
for _ in range(t):  
    if miller_rabin_single(n) == "composite":  
        return "composite"  
return "possible prime"
```

4.1 Результаты

Пример запуска (для $n = 131$ и $t = 20$):

Результат запуска программы изображен на Рисунок 4.1.

```
введи n: 131  
введи t: 20  
тест Ферма:           possible prime  
тест Соловья-Штрассена: possible prime  
тест Миллера-Рабина:  possible prime
```

Рисунок 4.1: Результат работы программы

5 Выводы

В ходе выполнения лабораторной работы реализованы четыре вероятностных алгоритма проверки чисел на простоту: тест Ферма, вычисление символа Якоби, тест Соловья–Штрассена и тест Миллера–Рабина. Программа позволяет запускать тесты для заданного числа n с заданным числом раундов t и получать заключение «составное» или «вероятно простое».

Список литературы

1. *Crandall R., Pomerance C.* Prime Numbers: A Computational Perspective. — 2-е изд. — Springer, 2005. — ISBN 0-387-25282-7.
2. *Menezes A. J., van Oorschot P. C., Vanstone S. A.* Handbook of Applied Cryptography. — CRC Press, 1996. — ISBN 0-8493-8523-7.
3. *Miller G. L.* Riemann's Hypothesis and Tests for Primality // Journal of Computer and System Sciences. — 1976. — Т. 13, № 3. — С. 300—317. — DOI: 10.1016/S0022-0000(76)80043-8.
4. *Rabin M. O.* Probabilistic Algorithm for Testing Primality // Journal of Number Theory. — 1980. — Т. 12, № 1. — С. 128—138. — DOI: 10.1016/0022-314X(80)90084-0.
5. *Solovay R. M., Strassen V.* A Fast Monte-Carlo Test for Primality // SIAM Journal on Computing. — 1977. — Т. 6, № 1. — С. 84—85.